

QA & Debugging Case Study

NextTech AI Support Bot

Functional regression, defect reproduction and fix verification for a customer-support web application.

Portfolio owner Alaa Madi	Report date 29 September 2026
Application FastAPI web chat & agent dashboard	Evidence scope Local, offline, development environment

62 Python tests passed	1 Dashboard regression test passed	81.66% Coverage of selected modules
----------------------------------	--	---

Executive assessment

The local support workflow passed the recorded checks. Four reproducible defects are documented here with before/after evidence: configuration loaded too late, unsafe dashboard HTML rendering, an incorrect dashboard status reference, and unnecessary escalation of greetings.

The fixes are traceable to a reviewable pull request. Remaining production risks are documented separately from verified fixes. This sample demonstrates structured QA reporting and technical debugging; it is not a mobile-device or production-load certification.

Evidence-backed result
Fresh evidence capture: 29 September 2026, 13:20 UTC.
Baseline: c595d48 → Tested fix: f38ca9d
Fix review: github.com/alaamadai/Chat-Bot/pull/32

Prepared from project source, executable regression checks and captured command output. Technical investigation, automation and report preparation used AI-assisted development tooling. Results describe this project and do not imply testing of a prospective client's application.

Scope and verification

The review follows the main support journey: create a signed chat session, send a message, escalate when appropriate, claim the conversation, reply as an agent and return the conversation to the bot.

Environment	Recorded configuration
Host / runtime	Windows · Python 3.13.12 · Node.js v24.19.0
Backend / persistence	FastAPI · SQLAlchemy · temporary SQLite database
AI / integrations	Offline LLM provider and local embeddings; live external calls excluded from the regression run
Dashboard execution	Node VM with simulated DOM and controlled API responses; not a browser/device matrix
Packaging	Python 3.12 Docker image successfully built during the preceding project review

Verification matrix

Check	Outcome	Evidence
Python regression suite	62 passed; 184 warnings	pytest.txt
Dashboard safety & state	1 test passed	dashboard.txt
Before/after defects	Original failures confirmed; corrected behavior confirmed	before.txt, after.txt, dashboard-comparison.txt
Syntax / undefined names	Ruff passed	ruff.txt
High-severity source scan	Bandit passed	bandit.txt
Clean-database migrations	7 migrations applied	migrations.txt

Evidence files are in docs/qa-evidence/. Coverage is 81.66% across the modules selected by CI, not the entire repository. The dashboard assertion covers both escaped rendering and the status update.

Previously verified local HTTP journey

Health, readiness, chat page, dashboard and API docs returned HTTP 200. Login, signed chat, greeting, handoff, claim, agent reply, transcript retrieval and return-to-bot checks passed during the preceding review. These results are recorded in docs/PROJECT_REVIEW.md; they are not a new device test.

Interpretation: Passing offline checks supports local functional confidence. It does not establish live AI quality, external delivery reliability, security certification or capacity under load.

Configuration and rendering

BUG-01 · Environment settings loaded after initialization

HIGH · P1

FIXED / RETESTED

Precondition: Fresh Python process; database and JWT settings supplied through a synthetic dotenv file rather than inherited environment variables.

Reproduce: 1. Set a distinct SQLite URL, signing key and token lifetime in the fixture. 2. Import `intake.main`. 3. Compare initialized database/auth values with the fixture.

Expected: Application initialization uses the configured values.

Actual before fix: All three comparisons returned false. The default database and authentication settings were captured before dotenv loading.

Impact / cause: Local configuration can silently target the wrong database or use an unintended signing key. Dotenv was loaded from the response generator after dependent modules initialized.

Resolution: Shared early configuration loader imported before environment-dependent initialization. After-fix comparisons all returned true.

Trace: `core/config.py`, `db/database.py`, `auth/web_session.py` · `tests/test_configuration.py` · Evidence: `before.txt` / `after.txt`.

BUG-02 · Queue data rendered as raw HTML

HIGH · P1

FIXED / RETESTED

Precondition: Controlled conversation response containing markup in a customer display field.

Reproduce: 1. Set `user_id` to `<img src=x onerror=alert(1)>`. 2. Invoke `renderQueue` in the dashboard harness. 3. Inspect the generated markup.

Expected: The supplied value is displayed as text, without creating an HTML element.

Actual before fix: The queue output contained a raw `<img>` element.

Impact / cause: An HTML injection sink in the agent interface creates potential script-execution risk when attacker-controlled data reaches it. API values were interpolated directly into `innerHTML`.

Resolution: Escape queue fields and message roles. Retest confirms raw markup is absent. The harness does not execute the payload or claim browser exploit verification.

Trace: `dashboard/index.html` · `tests/dashboard.test.cjs` · Evidence: `dashboard-comparison.txt`.

Agent state and handoff

BUG-03 · Conversation status badge does not update

MEDIUM · P2

FIXED / RETESTED

Precondition: An agent opens a conversation whose API status is `WAITING_FOR_AGENT`; the global status property is a string.

Reproduce: 1. Initialize the dashboard harness with `status: ''`. 2. Return the conversation from the API stub. 3. Call `openConversation`. 4. Inspect the status element.

Expected: The visible badge reflects the API conversation status.

Actual before fix: The element was unchanged; `status_element_updated` returned false.

Impact / cause: Agents cannot reliably see conversation state. The implicit global reference resolves to `window.status` instead of the element.

Resolution: Use `document.getElementById('status')`. Retest returns true for the element update, independently of the global property.

Trace: `dashboard/index.html` · Evidence: `dashboard-comparison.txt`; verified in a simulated DOM.

BUG-04 · Standalone greeting triggers human handoff

MEDIUM · P2

FIXED / RETESTED

Precondition: An active bot conversation receives a standalone greeting such as `hi`.

Reproduce: 1. Classify `hi`. 2. Apply confidence scoring to the resulting intent. 3. Inspect `escalate_to_human`.

Expected: A recognized greeting remains with the bot; an actual support request still escalates.

Actual before fix: Intent was `general` with confidence 0.5, below the 0.6 handoff threshold. Escalation was true.

Impact / cause: Routine greetings unnecessarily enter the support queue. The classifier lacked a standalone greeting category.

Resolution: Match normalized standalone greetings after business intents. The retest reports `greeting` and `escalation` false. Regression cases cover Arabic/English greetings, mixed greeting/support messages and return-to-bot behavior.

Trace: `ai_brain/intent_classifier.py` · `tests/test_greeting_handoff.py` · Evidence: `before.txt` / `after.txt`.

The greeting and status fixes were already present locally at the start of the original review; they were preserved, verified and included in PR #32.

Traceability and next steps

Observed before → after

Controlled check	Baseline	Fixed
Configured database used	false	true
Configured signing key used	false	true
Configured token lifetime used	false	true
Raw HTML in queue output	true	false
Status element updated	false	true
Standalone greeting escalates	true	false

Synthetic data only. These are extracted test observations, not fabricated UI screenshots. Evidence is captured by the versioned scripts below.

```
python scripts/capture_qa_evidence.py
node scripts/compare_dashboard.cjs
```

Run from the repository with development dependencies installed and Node.js available. The capture script re-executes the offline suite, compares baseline and current code, and writes evidence under docs/qa-evidence/.

Remaining release risks — source-review findings

- **Outbound reliability:** some WhatsApp paths bypass the durable outbox; verify crash recovery and retries.
- **Access and runtime policy:** validate account disablement/token revocation and required-Redis configuration.
- **Conversation quality:** pass bounded history to generation, define Arabic behavior, and report real CRM outcomes.

Proposed next testing phase — not yet executed

Agree on target devices, browser versions and latency/error budgets. Test mobile layouts, interrupted sessions, repeated submissions and SSE reconnects. Then run controlled baseline, ramp-up, spike and soak scenarios on staging with realistic services; record concurrency, request rate, p50/p95/p99 latency, error rate and resource use. Retest failed journeys after fixes.

Release recommendation: Suitable for local demonstration and further QA. Production sign-off remains pending live integrations, PostgreSQL/Redis checks, real-browser/device coverage and measured load testing. No capacity or mobile compatibility claims are made.

Verification references: [Fix PR #32](#) · [Tested code commit](#) · docs/PROJECT_REVIEW.md · raw evidence and capture scripts accompany this report.